MOVEINSYNC
COMMUTE TRUST REVIEW

FIELD NOTE 01 - EMPLOYEE EXPERIENCE

# Fix commute trust before polishing the MoveInSync app

The fastest credible path to a higher rating is to make the commute dependable at its two highest-stakes moments: opening the app and trusting the trip state. This report turns public Play Store evidence into a 30-day engineering plan.

PUBLIC GOOGLE PLAY EVIDENCE THROUGH 9 OCT 2026 / 910 UNIQUE REVIEWS



EXECUTIVE SUMMARY

# Restore trust before redesigning the surface

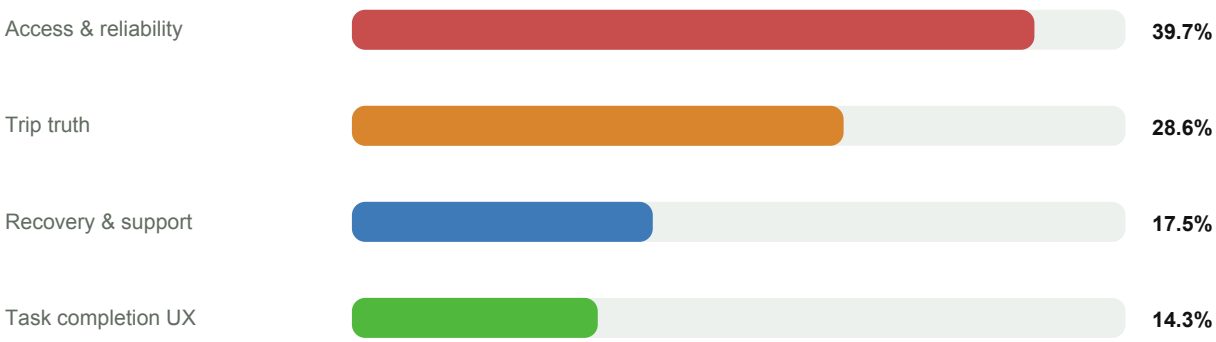
Access and reliability is the first fix. It represents 39.7% of the 63 recent manually coded complaints. Crashes, loading failures and login/OTP failures prevent every other commute task.

Trip truth is the second fix. Tracking, route, notification and boarding-state complaints contribute another 28.6%. Together, these two groups account for 68.3% of the recent coded complaint set.

In the 600-review newest-sort sample, the mean was 3.77 and 29.3% were one or two stars. These are sample signals, not the Play Store's official aggregate rating.

| 3.77 / 5 | 29.3% | 68.3% |
| --- | --- | --- |
| Newest-review sample | One-to-two-star share | Top two complaint groups |

**Share of 63 recent manually coded complaints**

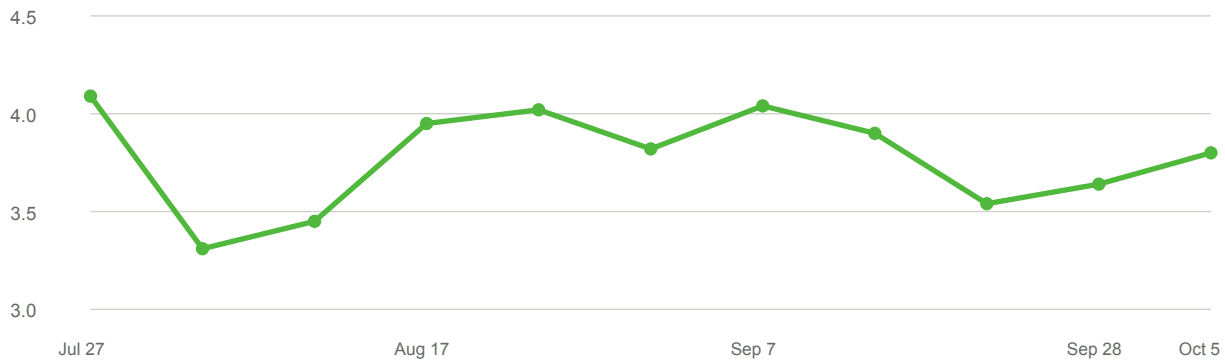



**Decision: ship release health gates first, then a canonical Trip Assurance surface, then structured recovery. Do not begin with a broad visual redesign.**

PUBLIC EVIDENCE

# Ratings move; the trust failures repeat

### Weekly average in the 600 newest-review sample




The weekly sample is volatile and should not be overinterpreted. The complaint language is more decision-useful: employees repeatedly describe being unable to access the app or unable to trust the booking, ETA, route and boarding state presented.

A keyword classifier was built and validated, but achieved only 43.3% exact primary-theme agreement on a frozen 30-review set. Automated theme totals were rejected; the recommendation uses manually coded evidence.

## Version signal

| Version | Reviews | Avg rating | 1-2 star share |
|---|---|---|---|
| 12.16.0 | 67 | 3.85 | 26.9% |
| 12.16.1 | 38 | 3.53 | 34.2% |
| 12.17.0 | 141 | 3.76 | 29.8% |
| 12.18.0 | 65 | 4.11 | 20.0% |
| 12.18.1 | 94 | 3.90 | 26.6% |
| 12.19.0 | 102 | 3.58 | 35.3% |

Version comparisons are descriptive, not causal. Versions overlap in time and 9.2% of the full deduplicated dataset has no reported version. Pair this view with internal crash and journey telemetry before rollback decisions.

PRIORITIZED BUILD PLAN

# What engineering should start tomorrow

**P0**

Make access boringly reliable
39.7% of recent coded complaints.

Build next: Crash/startup telemetry; login and OTP funnel with failure codes; secure session recovery; retry/fallback states; staged rollout and rollback switch.

Know it worked: Proposed SLOs: crash-free sessions at least 99.5%; core screen loads within 5s at least 99%; login/OTP completion at least 98%. Calibrate against internal baselines.

**P0**

Create one source of trip truth
28.6% of recent coded complaints.

Build next: Idempotent booking and cancellation writes; canonical trip-state contract; server reconciliation; cached last-confirmed trip; timestamped GPS freshness; explicit stale and mismatch states.

Know it worked: Booking reconciliation; status mismatches per 1,000 active trips; P95 location freshness; ETA error by version and tenant.

**P1**

Turn failures into traceable recovery
17.5% of recent coded complaints.

Build next: Structured issue categories; automatic trip/app/device context; visible case ID and status; operations-versus-product routing; Play review response workflow.

Know it worked: Support contacts per 1,000 trips; acknowledgement and resolution time; repeat contacts; developer response rate.

**P2**

Remove task friction
14.3% of recent coded complaints.

Build next: Skippable feedback; explicit waitlist and confirmation; booking/cancellation receipt; contextual safety prompts; no-show evidence and dispute path.

Know it worked: Task completion; forced-feedback dismissals; no-show disputes; median time to complete core tasks.

PRODUCT CONCEPT

# Trip Assurance: one surface for the facts that matter

The interactive HTML submission demonstrates these three states with an automatic preview and user-controlled switching. The product decision is the state contract, not merely the visual treatment.

OPEN THE INTERACTIVE DASHBOARD INCLUDED WITH THIS SUBMISSION

| NORMAL | STALE GPS | SYSTEM MISMATCH |
| --- | --- | --- |
| Booking, cab, pickup time, tracking and OTP agree. Show live ETA with an update timestamp. | Keep the confirmed booking, label the last update, and never present a guessed ETA as live. | Preserve the last confirmed fact, admit the conflict, open a diagnostic-rich case, and show its ID. |

## Core contract

**booking_state + allocation_state + pickup_time + vehicle + location_age + eta_confidence + boarding_state + recovery_case**

## Rules

- A booking mutation must be idempotent.
- Contradictory sources must resolve through a canonical state machine.
- Every live value carries freshness and confidence.
- Offline or stale states retain only the last confirmed fact.
- Recovery automatically includes trip and app diagnostics.
- Safety actions remain available even when other data is degraded.

30-DAY DELIVERY

# Sequence work around learning and release safety

| Days | Outcome | Work |
|---|---|---|
| 1-3 | **A shared failure view** | Instrument launch, authentication, booking retrieval, allocation, tracking freshness, OTP and support handoff. Cut by version, OS, device, network and tenant. |
| 4-10 | **Stable access** | Fix largest crash/authentication clusters. Add session recovery and OTP failure states. Pause staged rollout on SLO regression. |
| 8-18 | **Trip Assurance pilot** | Ship the canonical view model, booking reconciliation and stale-state handling behind a feature flag to one enterprise cohort with a comparable holdout. |
| 15-24 | **Traceable recovery** | Attach diagnostics, expose case status and route operational versus technical failures. |
| 25-30 | **Scale decision** | Review critical journey success, trust failures, support contacts, low-star review rate and guardrails. Expand only on supported improvement. |

## Measurement

Primary: critical journey success and trust failure rate per 1,000 active trips.
Lagging: one-to-two-star reviews per 10,000 completed trips, tagged to access or trip truth.
Guardrails: false no-show disputes, SOS availability, battery/data use and support contacts.

METHODS AND LIMITS

# A reproducible pipeline with explicit uncertainty

I used Node.js and the open-source google-play-scraper package to collect 600 newest and 400 helpful English-language reviews for India, then deduplicated them by review ID. Saved data excludes public reviewer names and profile images.

I manually coded 160 reviews with one primary theme using a documented taxonomy. Of these, 118 were specific complaints; 63 fell inside the recent 90-day window. The manual set is intentionally complaint-heavy, so theme shares indicate direction rather than population prevalence.

The 910-review decision baseline joins newest and helpfulness-ranked samples and is not a random sample. Weekly rating uses only the 600 newest reviews. Version analysis is descriptive. Google Play may display different aggregate ratings by locale or device.

A frozen keyword classifier evaluation reached 43.3% exact primary-theme accuracy. Its aggregate output is excluded. A production monitoring loop should use a stronger semantic classifier, human review of low-confidence cases and periodic frozen-set evaluation.

The submitted repository implements the recurring public monitor: GitHub Actions refetches the quantitative layer every Monday at 07:47 IST, validates it, rebuilds the dashboard and triggers the connected Vercel deployment. The 160 human-coded records remain a visibly dated snapshot until another human review pass.

## Reproducible commands

| | |
|---|---|
| npm run fetch:reviews | Collect and deduplicate current public reviews |
| npm run build:manual-analysis | Combine audited human labels and calculate themes |
| npm run analyze:reviews | Generate quality, time and version summaries |
| npm run refresh:data | Run the complete recurring refresh and validation pipeline |
| npm run build:dashboard | Produce the portable one-file interactive dashboard |

## Tools used

Node.js and JavaScript for collection and analysis; google-play-scraper for public reviews; manual coding with a documented taxonomy; Chart.js and vanilla JavaScript for the portable dashboard; ReportLab for this PDF; and AI-assisted coding and visual iteration with all quantitative claims checked against saved outputs.

## Next evidence to request

Internal crash traces, authentication/OTP funnel events, booking-state reconciliation logs, GPS freshness and ETA error, enterprise-tenant splits, completed-trip denominators, support categories and no-show disputes. These will verify root causes and calibrate the proposed SLOs before broad rollout.